Lab6: Let's Play DDPM

311351009 劉承霈

I. Introduction

In this lab, I work on generating synthetic images using the conditional DDPM method according to the multi-label conditions. The DDPM model is a state-of-the-art generative model. It generates a picture by sampling from Gaussian noise and denoising step by step. The overall procedure is that we first sample some data from the dataloader, and generate a sequence of t using a uniform distribution. After that, Gaussian noise is utilized as the ground truth for learning and evaluating whether the model has learned the denoising process. In the sampling stage, we begin with a Gaussian noise and utilize our model to predict and remove the noise until the image becomes clear. It's like cleaning up the picture little by little until we get the final result.

II. Implementation Details

1. Describe how you implement your model, including your choice of DDPM, UNet architectures, noise schedule, and loss functions.

- **UNet architecture**:

```
model = MyConditionedUNet(
    sample_size = 64,
    in_channels = 3,
    out_channels = 3,
    layers_per_block = 2,
    block_out_channels = (64, 128, 256, 256, 512, 512),
    down_block_types = (
        "DownBlock2D",
        "DownBlock2D",
        "DownBlock2D",
        "DownBlock2D",
        "AttnDownBlock2D",
        "DownBlock2D",
    ),
    up_block_types=(
        "UpBlock2D",
        "AttnUpBlock2D",
        "UpBlock2D",
        "UpBlock2D",
        "UpBlock2D",
        "UpBlock2D",
    ),
    class_embed_type = "timestep")
```

- In DDPM's denoising process, noise prediction is a crucial step, and this is accomplished using the adopted UNet architecture found in most DDPM implementations. Therefore, I import UNet2DModel from Hugging face diffuser, which is referenced from the link: https://huggingface.co/docs/diffusers/v0.16.0/en/api/models#diffusers.UNet2DModel

- The original diffusers' UNet2DModel only provides a single class label condition, but this task requires a multi-label condition, so I modify the original UNet2DModel: I add an embedding layer to convert the input condition vector to the same size as the time-embedding layer.

- **Noise Schedule**:

```
noise_scheduler = DDPMScheduler(num_train_timesteps = 1000, beta_schedule = 'squaredcos_cap_v2')
```

- I import DDPMScheduler from Hugging face diffuser, and use the cosine noise schedule, since it get better score than the linear one.

- **Loss function**:

```
self.loss_fn = nn.MSELoss()
loss = self.loss_fn(predict_noise, noise)
```

- MSELoss() is used to evaluate the difference between the original noise and the noise predicted by UNet.

- **DDPM**:

```
noise = torch.randn_like(image).to(self.device)
timesteps = torch.randint(0, 999, (image.shape[0],)).long().to(self.device)
noise_image = self.noise_scheduler.add_noise(image, noise, timesteps)
predict_noise = self.model(noise_image, timesteps, label).sample
```

- During each training iteration, noise is randomly generated, and timesteps are randomly sampled. These generated noise and time steps are then combined and passed into *noise_scheduler.add_noise()*. The noise scheduler adjusts the noise intensity based on the provided time steps, resulting in a noisy image after adding the noise to the original input.
- Then, we pass the noisy images, timesteps, and condition into the ConditionalUNet we wrote before, and it will help calculate the noise of the current image and send back the predicted noise.

- Last, we use MSELoss() to calculate the difference between the original sample noise and the predicted noise, we would like the predicted noise to be as close to the real noise as possible.

2. Specify the hyperparameters (learning rate, epochs, etc.)
   - learning rate: 1e-4
   - batch size: 32
   - epochs: 100
   - optimizer: AdamW
   - timestep: 1000
   - layers per block: 2

III. Results and discussion

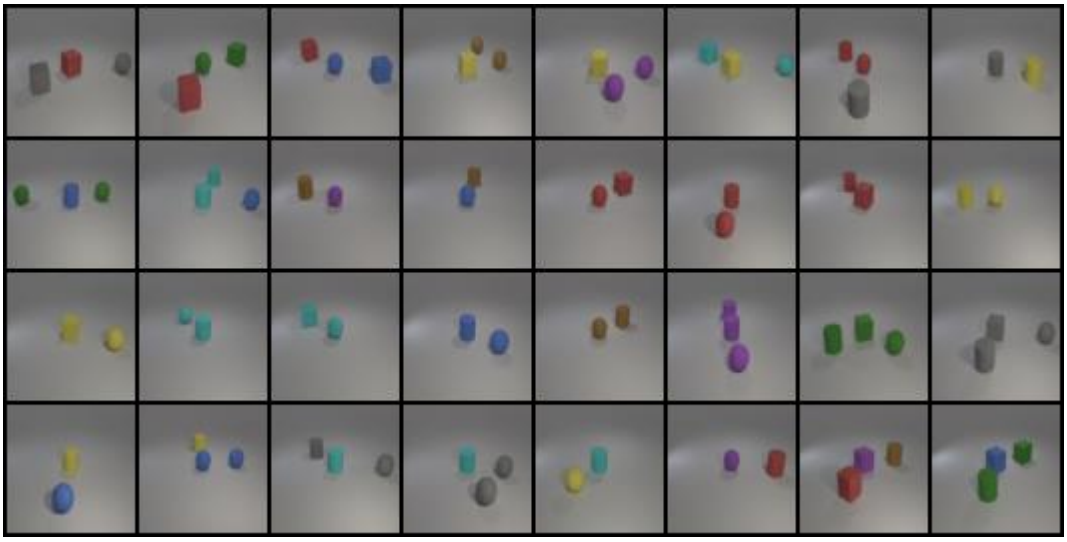
1. Show your accuracy screenshot based on the testing data.

```
Test: 0.9444444444444444
New Test: 0.8690476190476191
```

2. Show your synthetic image grids and a progressive generation image.
   - test.json:



- new_test.json:



3. Discuss the results of different model architectures or methods.

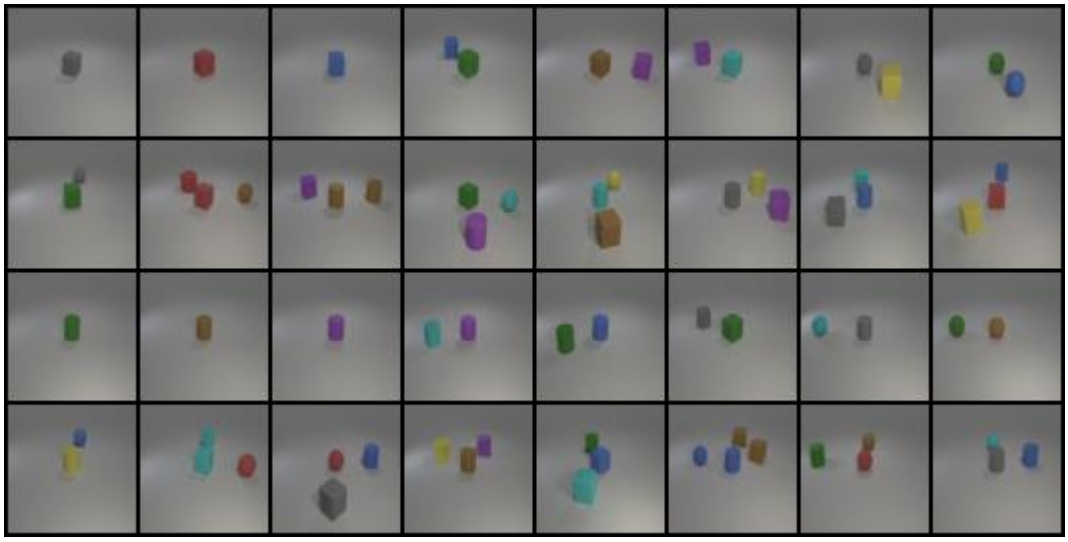
- **Different beta schedule**: At first, I apply the linear noise schedule.

  Although it also gets good results, and the generated synthetic images look great, the cosine noise schedule still gets better accuracy.

  - Accuracy applying linear noise schedule:

    ```
    Test: 0.8472222222222222
    New Test: 0.8333333333333334
    ```

  - test.json:



- new_test.json:

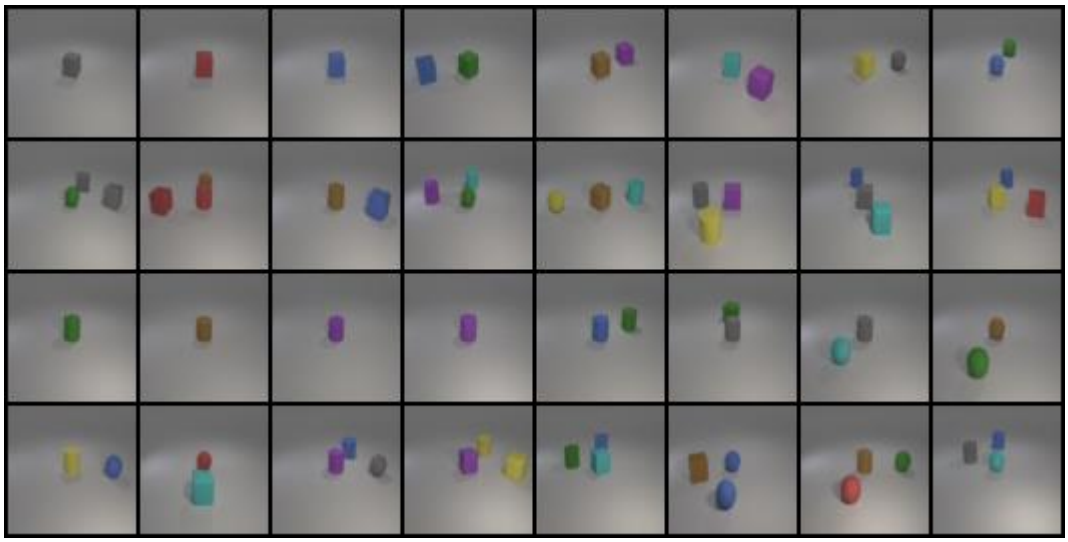


- **Different layers per block in UNet**: The original UNet used 2 layers per block, so we simplified the model to see how well the training worked. We can see that even with only one layer per block, the model still produces good results and both tests also get good score.

  - Accuracy applying 1 layer per block:

    ```
    Test: 0.8472222222222222
    New Test: 0.8690476190476191
    ```

  - test.json:



  - new_test.json:

